

RippleLab Day 06 Progress Report

Day: 06 of 30

Date: 2026-08-14

Status: Complete

Branch: codex/day-06-calculations

Implementation commit: aae9791

Deployed commit: aae9791

Production version: 2

Quality gate: Full local suite passed

1. Objective

Build the deterministic money-and-rate foundation for RippleLab. Day 6 replaces illustrative loan and deposit arithmetic with versioned, Decimal-safe primitives that can later power personal repo-rate scenarios without delegating calculations to an LLM.

2. Acceptance Criteria

- [x] Calculate an amortizing-loan EMI from principal, annual rate and term.
- [x] Calculate the outstanding loan balance after a valid number of payments.
- [x] Model partial or complete transmission of a policy-rate shock.
- [x] Recalculate a floating-rate loan payment after a rate reset.
- [x] Calculate simple fixed-deposit interest for an explicit term and day-count basis.
- [x] Reject negative money, invalid terms, unsupported day-count bases and out-of-range rates.
- [x] Return a formula version and explicit assumptions from every calculation.
- [x] Round money once, to the nearest paise, with half-up rounding.
- [x] Document units, formulas, examples and validation boundaries.
- [x] Show Day 6 progress and known-value fixtures on the deployed progress page.

3. Completed Work

The economic engine now exposes five pure calculation functions: EMI, outstanding balance, linear rate pass-through, floating-rate reset and simple deposit interest. Inputs use integer paise, integer basis points and whole days or months at the public boundary. Internal operations use Python `Decimal` with 50 digits of precision.

Each function returns a typed immutable result. The result includes its formula version and assumptions, making the output inspectable and reproducible. Money is not rounded during intermediate operations; the engine rounds the final monetary value exactly once to paise using `ROUND_HALF_UP`.

The public progress page now reports 6 of 30 days complete, identifies Day 7 as the next checkpoint and displays four known-value fixtures. A new causal strip explains the Day 6 repo-rate path: policy shock, pass-through assumption, revised annual rate and recalculated EMI.

4. Production Deployment

The Day 6 build is live at ripplelab-progress.nayaksiddhartha397.chatgpt.site. Sites production version 2 was saved and deployed from the exact pushed source commit aae9791. The deployment completed successfully on 2026-08-14.

Access remains owner-only during development. The landing page, progress page, design system and other read-only product surfaces can be inspected through the authenticated Codex/ChatGPT session. Hosted account creation and persistent financial profiles still require Supabase production environment values; no placeholder secrets were committed.

5. Numeric Policy

Concern	Rule	Reason
Money input	Integer paise	Prevent ambiguous floating-point currency input
Rate input	Integer basis points	Make rate units exact and reviewable
Internal arithmetic	Decimal, precision 50	Avoid binary floating-point drift
Money output	Round once to paise	Keep formulas stable and auditable
Rounding mode	Half up	Match ordinary financial presentation
Rate-change output	Quantized to 0.0001 bps	Preserve partial pass-through detail

Boolean values are explicitly rejected even though Python normally treats `bool` as a subtype of `int`. This prevents accidental values such as `True` from becoming a one-paise principal or a one-month term.

6. Formula Ledger

6.1 Amortizing-loan EMI

Formula version: `loan.emi.v1`

```
monthly_rate = annual_rate_bps / 10000 / 12
EMI = principal * monthly_rate * (1 + monthly_rate)^months
      / ((1 + monthly_rate)^months - 1)
```

For a zero-rate loan, EMI is principal divided by the term. The final result is rounded once to paise.

6.2 Outstanding balance

Formula version: `loan.outstanding_balance.v1`

```
balance_after_k = principal * (1 + monthly_rate)^k
                  - EMI * ((1 + monthly_rate)^k - 1) / monthly_rate
```

The function uses the unrounded contractual EMI internally. This avoids compounding presentation rounding through every simulated payment. At the full term, the balance is normalized to zero.

6.3 Linear rate pass-through

Formula version: `rates.linear_pass_through.v1`

```
effective_change_bps = policy_shock_bps * pass_through_ratio_bps / 10000
```

A pass-through ratio of 0 means no transmission; 10,000 means complete transmission. Negative policy shocks are supported, while the absolute shock is capped at 10,000 basis points.

6.4 Floating-rate reset

Formula version: `loan.floating_rate_reset.v1`

```
effective_change_bps = policy_shock_bps * pass_through_ratio_bps / 10000
new_annual_rate_bps = current_annual_rate_bps + effective_change_bps
new_EMI = EMI(outstanding_principal, new_annual_rate, remaining_months)
monthly_change = new_EMI - current_EMI
```

The model assumes immediate repricing after the supplied shock and a constant remaining term. Lender-specific spreads, reset dates, fees and benchmark floors are not yet modeled.

6.5 Simple deposit interest

Formula version: `deposit.simple_interest.v1`

```
gross_interest = principal * annual_rate_bps / 10000
                 * term_days / day_count_basis
```

Supported day-count bases are 360, 365 and 366. This primitive returns gross simple interest before tax and does not yet model compounding or premature-withdrawal penalties.

7. Known-Value Fixtures

Fixture	Input	Deterministic result
Home-loan EMI	INR 50,00,000; 8.50%; 240 months	INR 43,391.16 per month
Balance after 60 payments	Same loan; 60 payments	INR 44,06,359.16 outstanding
Floating reset	100 bps cut; 70% pass-through; 180 months left	Rate 8.50% to 7.80%; EMI lower by INR 2,189.36 per month
Deposit interest	INR 10,00,000; 7.25%; 365 days; actual/365	INR 72,500.00 gross interest

These values are executable regression fixtures, not predictions. The floating-rate example demonstrates the arithmetic under a stated 70% pass-through assumption; it does not claim that every lender will transmit 70% of an RBI move.

8. Validation Boundaries

The calculation module rejects:

- negative principal or deposit amounts;
- zero, negative or longer-than-1,200-month loan terms;
- payment counts outside zero through the contractual term;
- annual rates below zero or above 10,000 basis points;
- rate shocks whose absolute value exceeds 10,000 basis points;
- pass-through ratios below zero or above 10,000 basis points;
- floating reset outcomes outside zero through 10,000 basis points;
- negative deposit terms, terms longer than 36,600 days and unsupported day-count bases.

Valid boundary behavior is also tested: zero-rate loans, zero principal, zero deposit term, no pass-through, complete pass-through, and a fully paid loan.

9. Test and Verification Evidence

Check	Method	Result
Calculation unit tests	Known values, boundaries and invalid inputs	27 passed
Complete API tests	Contract and calculation tests	33 passed
Python quality	Ruff lint and format verification	Passed
Canonical contracts	Schema, golden fixtures, units and references	Passed
Profile security	Migration, ownership, RLS, money and consent assertions	Passed
Web quality	ESLint, generated route types, TypeScript and production build	Passed
Browser suite	Desktop and mobile Chromium	19 passed; 3 intentional device skips
Accessibility	axe-core on public and account surfaces	No serious or critical violations
Secret scan	Repository scan	Passed
Full quality gate	UV_CACHE_DIR=/tmp/ripplelab-uv-cache pnpm check	Passed
Hosting build	Vinext/Cloudflare Worker production bundle	Passed
Production deployment	Owner-only Sites version 2	Succeeded

The API suite reports one dependency deprecation warning from FastAPI's Starlette test client regarding `httplib`. It does not affect the 33 passing tests and is tracked as dependency maintenance rather than a Day 6 functional failure.

10. Progress and Sharing Experience

The `/progress` page now includes a four-card formula ledger with the known EMI, balance, reset and deposit fixtures. Its completion indicator moved from 5/30 to 6/30 and the next action points to the first end-to-end repo-rate scenario on Day 7.

A site-specific 1536 by 1024 social preview card was added. It visualizes the causal path from RBI rate to loan rate to personal EMI and names the four completed Day 6 calculation areas. Open Graph and Twitter metadata use the incoming request host to construct an absolute image URL, so the same build remains correct on its Sites hostname and in local development.

11. Important Files

- `services/economic-engine/src/ripplelab_engine/calculations.py` - versioned deterministic primitives and result types.
- `services/economic-engine/tests/test_calculations.py` - 27 calculation tests.
- `docs/CALCULATION_PRIMITIVES_V1.md` - formula, units, assumptions and validation reference.
- `services/economic-engine/README.md` - engine capability summary.
- `apps/web/src/app/progress/page.tsx` - Day 6 progress and formula ledger.
- `apps/web/src/app/globals.css` - responsive formula-ledger presentation.

- `apps/web/src/app/layout.tsx` - request-aware social sharing metadata.
- `apps/web/public/ripplelab-day-06-social.png` - Day 6 social preview card.
- `apps/web/package.json` - stable Next.js type-check command using `next typegen`.
- `CHANGELOG.md` and `README.md` - repository-level Day 6 status.

12. Engineering Decisions

- Keep all economic calculations deterministic and reserve the LLM for parsing and explanation.
- Use integer paise and basis points at system boundaries.
- Use `Decimal` throughout internal arithmetic and round final money once.
- Version every formula independently so saved simulations can be reproduced after later model changes.
- Return assumptions with the result rather than hiding them in implementation comments.
- Use a separate rate-transmission primitive because policy-rate movement and borrower-rate movement are not identical.
- Keep the outstanding-balance formula independent so Day 7 can reset only the remaining principal and term.
- Treat gross deposit interest separately from taxation, compounding and penalties.
- Generate Next.js route types before TypeScript checking to prevent adapter builds from leaving stale route declarations.
- Keep the production preview owner-only until sharing is a deliberate product decision.

13. Risks and Current Limitations

The rate pass-through model is linear and assumption-driven. Actual bank transmission can vary by benchmark, loan contract, lender, borrower and reset date. Day 7 must surface that assumption prominently and must not present the result as a guaranteed RBI-to-EMI mapping.

The deposit primitive models gross simple interest. Compounding schedules, tax deducted at source, marginal tax effects and early withdrawal remain out of scope for this formula version.

The floating-rate reset keeps the remaining term constant. A later product option may let the user compare a lower EMI against retaining the EMI and shortening the term.

Hosted authentication and profile persistence remain unavailable until Supabase production variables are configured. The deployed progress and read-only pages are usable now.

No code blocker remains for Day 6.

14. Day 7 Handoff

Day 7 should connect the Day 5 repo-rate contract to the Day 6 primitives in one orchestrated scenario. It should accept a valid profile snapshot and repo-rate shock, calculate loan and deposit impacts, produce annual net impact, build a causal graph whose edges reference assumptions, and return a confidence breakdown. An API test should reproduce the Day 6 known values through the complete scenario boundary.

Sign-off

Prepared by: Codex | **Evidence reviewed:** Yes | **Ready to continue:** Yes